



Option pricing in the Heston model with physics inspired neural networks

Donatien Hainaut¹ · Alex Casas²

Received: 31 January 2024 / Accepted: 5 August 2024 / Published online: 26 August 2024

© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2024

Abstract

In absence of a closed form expression such as in the Heston model, the option pricing is computationally intensive when calibrating a model to market quotes. this article proposes an alternative to standard pricing methods based on physics-inspired neural networks (PINNs). A PINN integrates principles from physics into its learning process to enhance its efficiency in solving complex problems. In this article, the driving principle is the Feynman-Kac (FK) equation, which is a partial differential equation (PDE) governing the derivative price in the Heston model. We focus on the valuation of European options and show that PINNs constitute an efficient alternative for pricing options with various specifications and parameters without the need for retraining.

Keywords Neural networks · Options · Heston model · Feynman-Kac equation

JEL Classification C6 · G1

1 Introduction

A physics-inspired neural network (PINN) incorporates principles from physics into its learning process, enhancing its efficiency in solving complex scientific problems. Compared to existing approaches, PINNs reduce the reliance on large datasets and provide accurate predictions even with sparse or noisy measurements. Researchers have employed PINNs to address a diverse range of problems, including fluid dynamics, solid mechanics, heat transfer, and quantum mechanics. For instance, Mao et al. (2020) solved with PINN's, Euler equations modeling high-speed aerodynamic flows. Jin et al. (2021) solved the incompressible Navier–Stokes equations. Cai et al. (2021) use PINN's for analyzing heat flow convection. For a recent survey on developments

✉ Donatien Hainaut
donatien.hainaut@uclouvain.be

Alex Casas
a.casas@detralytics.eu

¹ UCLouvain- LIDAM, Voie du Roman Pays 20, 1348 Louvain-la-Neuve, Belgium

² Detralytics, Rue Réaumur, 124, 75002 Paris, France

and applications of PINNs, we refer to Cuomo et al. (2022) for a comprehensive literature review.

In this study, we demonstrate that PINNs can be used for the valuation of options in the Heston model. In this framework, the stock price is governed by a geometric Brownian motion with stochastic volatility. Options are valued either by discrete Fourier transform (FFT) or by Monte Carlo simulations. During the calibration phase, the parameters of the Heston model are adjusted to best replicate market quotes of options. This step requires multiple option valuations and is therefore computationally intensive. To expedite the process, we propose a neural network model that can instantaneously price options with various features and parameters. The network is trained to solve the Feynman-Kac equation in multiple configurations and is called PINN for this reason.

In summary, a PINN is an approximate solution of a non-linear partial differential equation (PDE) that describes the dynamics of a model. The neural network is trained by minimizing the approximation error at sampled points within the PDE domain and on its boundaries. This method originates from the work of Lee and Kang (1990) and has since been applied to various non-linear PDEs. For example, Raissi et al. (2019) used PINNs for solving two main classes of mathematical problems: data-driven solutions and data-driven discovery of partial differential equations. In the field of physics, Carleo and Troyer (2017) and Cai (2018) employed PINNs to accurately approximate quantum many-body wave functions.

The literature on the valuation of financial derivatives using neural networks is relatively recent. Hejazi and Jackson (2016) developed a neural network to price and estimate the 'Greeks' for a large portfolio of variable annuities. Doyle and Groendyke (2019) priced and hedged equity-linked contracts using neural networks. They constructed a dataset of variable annuity prices with various features through Monte Carlo simulations. The neural network was then estimated by minimizing prediction errors using a scaled conjugate gradient descent. Sirignano and Spiliopoulos (2018) introduced a Deep Galerkin Method (DGM) based on a network architecture inspired by long short-term memory (LSTM) neural cells. Their approach was tested on a class of high-dimensional free boundary partial differential equations (PDEs) and on high-dimensional Hamilton-Jacobi-Bellman PDEs and Burgers' equation. Gatta et al. (2023) evaluated a suitable PINN for the pricing of American multi-asset options and proposed a novel algorithmic technique for free boundary training. Al-Aradi et al. (2022) extended the DGM in several directions to solve Fokker-Planck and Hamilton-Jacobi-Bellman equations. More recently, Horvath et al. (2021) demonstrated, under mild assumptions, the convergence of the Deep Galerkin and PINNs method for solving PDEs. Nonetheless, the convergence rate remains unknown and depends upon the studied problem. Glau and Wunderlich (2022) formalized and analyzed the deep parametric PDE method for solving high-dimensional parametric partial differential equations. Hainaut (2024) adapts the PINNs method to evaluate guaranteed minimum accumulation benefits for various investment strategies with stochastic mortality and interest rates.

A close alternative to PINN is developed in Weinan et al. (2017), where they solve parabolic PDEs in high dimensions using neural networks connected to BSDEs. This approach employs two separate networks for the solution and its gradient. They apply

their algorithm to price options in a multivariate Black and Scholes model. Beck et al. (2019) introduce a similar method for solving high-dimensional PDEs based on a connection between fully nonlinear second-order PDEs and second-order backward stochastic differential equations. They illustrate the accuracy of their method with the Black and Scholes and Hamilton-Jacobi-Bellman equations. Using this principle, Barigou and Delong (2022) evaluate equity-linked life insurances with neural networks by solving a backward stochastic differential equation (BSDE). Unlike PINNs, BSDE methods rely on two networks instead of one. These networks respectively approximate the contract value and its gradient. This avoids convergence issues caused by the numerical calculation of gradient but any modification of contract specifications requires retraining.

Another valuation approach based on neural networks relies on simulations. Buehler et al. (2019) present a framework for pricing and hedging derivatives using neural networks that are trained on simulated sample paths of risk factors. In this case, the neural network approximates the optimal investment strategy, and the value is determined by averaging discounted payoffs obtained through simulations. Similarly, Horvath et al. (2021) studied the performance of deep hedging under rough volatility models, while (Biagini et al. 2023) developed a neural network approximation using simulations for the superhedging price and replicating strategy. As with BSDE approaches, any modification of contract specifications requires retraining. For a comprehensive review of algorithms based on neural networks for stochastic control and PDEs in finance, we recommend referring to Germain et al. (2021).

In the financial and insurance industry, four dominant numerical methods are commonly used for pricing contingent claim contracts: Monte-Carlo (MC) simulations, bi- or trinomial trees, PDE solving, or Fast Fourier Transform (FFT) inversion. As the calibration of a model needs multiple valuations of options with different features, these methods are computationally intensive. In contrast, the valuation using a PINN, once it is trained, becomes nearly instantaneous. Another noteworthy feature is the ability to parameterize a PINN with model parameters and contract features such the maturity. This provides a quick solution pricing without the need to retrain the network for each setting. This presents a significant advantage compared to BSDE or deep hedging approaches.

This article makes the following contributions. Firstly, we introduce a specific type of neural network in which intermediate layers are fed with both the output of the previous layer and the initial input vector. Such a network better captures the non-linear behavior of option prices compared to classical feed-forward networks. Secondly, we develop a scaled and centered version of the Feynman-Kac equation ruling option prices in the Heston model. Centering and scaling inputs avoids convergence issues related to vanishing gradients. In Hainaut (2024), model parameters are fixed and the network must be retrained after each modification of them. In this article, we train the network for various maturities of options and for a wide range of Heston parameters. In this sense, our model offers a significant flexibility compared to BSDE, Hainaut (2024) or deep hedging methods, which require retraining for each specific setting of Heston parameters.

2 Heston model in a nutshell

We consider a financial market composed of two underlying assets. The cash account earns a constant risk free rate r . The stock price, denoted by $(S_t)_{t \geq 0}$, is ruled by a geometric Brownian diffusion with a stochastic variance process, $(V_t)_{t \geq 0}$. These price and variance processes are defined on a complete probability space $(\Omega, \mathcal{F}, \mathbb{P})$ associated to two independent Brownian motions under $\mathbb{P}$, noted $\tilde{\mathbf{W}}_t = (\tilde{W}_t^{(1)}, \tilde{W}_t^{(2)})_{t \geq 0}^\top$. The state variables (S_t, V_t) are driven by the following stochastic differential equations (SDE's):

$$d \begin{pmatrix} S_t \\ V_t \end{pmatrix} = \begin{pmatrix} (r + \nu_S \sqrt{V_t}) S_t \\ \kappa \left(\gamma - \nu_V \frac{\sigma \sqrt{V_t}}{\kappa} - V_t \right) \end{pmatrix} dt + \begin{pmatrix} S_t \sqrt{V_t} \Sigma_S^\top \\ \sigma \sqrt{V_t} \Sigma_V^\top \end{pmatrix} d\tilde{\mathbf{W}}_t, \quad (1)$$

where κ, γ and σ are in $\mathbb{R}^+$. Σ_S, Σ_V are vectors such that $\Sigma = (\Sigma_S^\top, \Sigma_V^\top)$ is the (upper) Choleski decomposition of the correlation matrix:

$$\Sigma = \begin{pmatrix} \Sigma_S^\top \\ \Sigma_V^\top \end{pmatrix} = \begin{pmatrix} \sqrt{1 - \rho^2} & \rho \\ 0 & 1 \end{pmatrix}, \quad \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix} = \Sigma \Sigma^\top,$$

where $\rho \in (-1, 1)$ is the correlation between the stock price and its variance. The parameters ν_S, ν_V tune the risk premiums $\left(\nu_S \sqrt{V_t} - \nu_V \frac{\sigma \sqrt{V_t}}{\kappa} \right)$, of price and variance processes. We assume that prices of risk, noted $\boldsymbol{\theta} = (\theta_1, \theta_2)^\top$, are constant, i.e. the Brownian motions with drift under $\mathbb{P}$,

$$dW_t^{(j)} = d\tilde{W}_t^{(j)} + \theta_j dt, \quad j = 1, 2.$$

are Brownian motions under the risk-neutral measure, $\mathbb{Q}$. To keep identical dynamics under $\mathbb{P}$ and $\mathbb{Q}$, the risk premium parameters, stored in a vector $\boldsymbol{\theta} = (\theta_1, \theta_2)^\top$ are such that

$$\begin{aligned} \nu_S &= \sqrt{1 - \rho^2} \theta_1 + \rho \theta_2, \\ \nu_V &= -\theta_2. \end{aligned}$$

Under the risk-neutral measure $\mathbb{Q}$, the stock price and variance processes are therefore ruled by the following SDE's:

$$d \begin{pmatrix} S_t \\ V_t \end{pmatrix} = \begin{pmatrix} r S_t \\ \kappa (\gamma - V_t) \end{pmatrix} dt + \begin{pmatrix} S_t \sqrt{V_t} \Sigma_S^\top \\ \sigma \sqrt{V_t} \Sigma_V^\top \end{pmatrix} d\mathbf{W}_t, \quad (2)$$

where $\mathbf{W}_t = (W_t^{(1)}, W_t^{(2)})_{t \geq 0}^\top$. In the next section, we recall the valuation equation ruling option prices.

3 Valuation equation, European payoffs

We consider a European financial derivative expiring at time T and promising a general payoff, $H(S_T)$, function of stock price and variance at maturity. The fair value, denoted by L_t is equal to the expected discounted cash-flows under the risk-neutral measure $\mathbb{Q}$:

$$L_t = \mathbb{E} \left(e^{-r(T-t)} H(S_T) \mid \mathcal{F}_t \right).$$

In the numerical illustration, we consider a put option with a strike price K . For such a derivative, the payoff has the following form

$$H(S_T) := (K - S_T)_+.$$

A physics-inspired neural network (PINN) integrates principles from physics, including partial differential equations (PDEs) governing the behavior of state variables. In a financial context, we adopt the Feynman-Kac (FK) equation as our guiding principle. The FK equation is a PDE satisfied by all assets traded in an arbitrage-free market. In this section, we construct this equation for the valuation of European options in the Heston model. In the following section, we utilize a neural network to solve it for various market parameters and contract features. To lighten future developments, we denote the vector of state variables by

$$\mathbf{y}_t = (S_t, V_t)^\top.$$

Under the risk-neutral measure $\mathbb{Q}$, this multivariate process is ruled by the following SDE:

$$d\mathbf{y}_t = \boldsymbol{\mu}_y(t, \mathbf{y}_t)dt + \Sigma_y(t, \mathbf{y}_t)d\mathbf{W}_t,$$

where $\boldsymbol{\mu}_y(\cdot)$ is a vector of dimension 2 and $\Sigma_y(\cdot)$ is a 2×2 matrix:

$$\boldsymbol{\mu}_y(t, \mathbf{y}_t) = \begin{pmatrix} r S_t \\ \kappa (\gamma - V_t) \end{pmatrix}, \quad \Sigma_y(t, \mathbf{y}_t) = \begin{pmatrix} S_t \sqrt{V_t} \Sigma_S^\top \\ \sigma \sqrt{V_t} \Sigma_V^\top \end{pmatrix}.$$

We emphasize the dependence to parameters by denoting the contract price as follows:

$$L_t = V(t, \mathbf{y}_t \mid r, \kappa, \gamma, \sigma, \rho, T).$$

Under the risk-neutral measure $\mathbb{Q}$, all discounted traded securities, including derivatives are martingales to avoid arbitrages, i.e.:

$$\mathbb{E} \left(e^{-rs} L_s \mid \mathcal{F}_t \right) = e^{-rt} L_t. \quad (3)$$

Let us respectively denote the gradient and the Hessian of $V(\cdot)$ with respect to $\mathbf{y}$ by $\nabla_{\mathbf{y}} V$ and $\mathcal{H}_{\mathbf{y}}(V)$:

$$\nabla_{\mathbf{y}} V = \begin{pmatrix} \partial_S V \\ \partial_V V \end{pmatrix}, \quad \mathcal{H}_{\mathbf{y}}(V) = \begin{pmatrix} \partial_{SS} V & \partial_{SV} V \\ \partial_{SV} V & \partial_{VV} V \end{pmatrix}.$$

Applying the Itô's lemma to $e^{-rt} L_t = e^{-rt} V(\cdot)$ allows us to infer the dynamic of the the discounted option price:

$$\begin{aligned} d(e^{-rt} V) &= e^{-rt} \left(-rV + \partial_t V + \boldsymbol{\mu}_{\mathbf{y}}(t, \mathbf{y}_t)^\top \nabla_{\mathbf{y}} V \right) dt \\ &\quad + \frac{e^{-rt}}{2} \text{tr} \left(\Sigma_{\mathbf{y}}(t, \mathbf{y}_t) \Sigma_{\mathbf{y}}(t, \mathbf{y}_t)^\top \mathcal{H}_{\mathbf{y}}(V) \right) dt \\ &\quad + e^{-rt} \nabla_{\mathbf{y}} V \Sigma_{\mathbf{y}}(t, \mathbf{y}_t) d\mathbf{W}_t. \end{aligned}$$

The condition (3) is fulfilled only if the following equality, called Feynman-Kac (FK) equation, holds:

$$\begin{aligned} 0 &= \partial_t V - rV + \boldsymbol{\mu}_{\mathbf{y}}(t, \mathbf{y}_t)^\top \nabla_{\mathbf{y}} V \\ &\quad + \frac{1}{2} \text{tr} \left(\Sigma_{\mathbf{y}}(t, \mathbf{y}_t) \Sigma_{\mathbf{y}}(t, \mathbf{y}_t)^\top \mathcal{H}_{\mathbf{y}}(V) \right). \end{aligned} \quad (4)$$

In this case, the discounted price is indeed rewritten as

$$e^{-rs} L_s = e^{-rt} L_t + \int_t^s e^{-ru} \nabla_{\mathbf{y}} L_u \Sigma_{\mathbf{y}}(u, \mathbf{y}_u) d\mathbf{W}_u,$$

which is a martingale under mild conditions of integrability. The boundary constraint on $V(\cdot)$ at expiry is equal to

$$V(T, \mathbf{y}_T | r, \kappa, \gamma, \sigma, \rho, T) = H(S_T). \quad (5)$$

For most of current European payoffs, the option value if the stock price falls to zero is the discounted value of a constant payoff. In this case, we have a lower boundary constraint on $V(\cdot)$:

$$V\left(t, (0, V_t)^\top | r, \kappa, \gamma, \sigma, \rho, T\right) = e^{-r(T-t)} H(0). \quad (6)$$

we refer to Karatzas and Schreve (1998) Sect. 4.4 for a general presentation of the Feynman-Kac equation and to Friedman (1964), Chapter 1, Friedman (1975), p. 147 for conditions under which the Feynman-Kac equation admits a solution. In the Heston model, a sufficient condition is a continuous payoff $H(\cdot)$ function such that $\forall x \in \mathbb{R}^+$,

$$|H(x)| \leq L(1 + \|x\|^v),$$

for some constant $L, v > 0$.

We approximate the solution of the FK Eq. (4) using a neural network that takes as input the time, state variables, and parameters of the Heston model. The mathematical definition of a neural network is revisited in the following section, but it's worth noting that a neural network employs bounded activation functions, such as sigmoid or hyperbolic tangents. To mitigate convergence issues associated with the vanishing gradient problem, we standardize and scale the network's inputs. This preprocessing step slightly modifies the FK equation. Let us define the vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^2$ as follows

$$\begin{aligned}\mathbf{a} &= (a_S, a_V)^\top, \\ \mathbf{b} &= (b_S, b_V)^\top.\end{aligned}\quad (7)$$

$\mathbf{a}$ and $\mathbf{b}$ are chosen to normalize a random sample of state variables. This point is discussed in the next section. The vector of centered and scaled state variables is denoted by

$$\tilde{\mathbf{y}}_t = \mathbf{a} + \mathbf{b} \odot \mathbf{y}_t, \quad (8)$$

where $\odot$ is the elementwise product. In a similar manner, we center and scale the time with coefficients a_h and b_h that depend on the horizon of valuation. The centered scaled time and durations are defined as follows

$$\begin{cases} \tilde{t} &= a_h + b_h t, \\ \tilde{T} &= a_h + b_h T. \end{cases} \quad (9)$$

The normalized state variables, $\tilde{\mathbf{y}}_t = (\tilde{S}_t, \tilde{V}_t)$ are ruled by the SDE:

$$d\tilde{\mathbf{y}}_t = \boldsymbol{\mu}_{\tilde{\mathbf{y}}}(t, \tilde{\mathbf{y}}_t)dt + \Sigma_{\tilde{\mathbf{y}}}(t, \tilde{\mathbf{y}}_t)d\mathbf{W}_t,$$

where $\boldsymbol{\mu}_{\tilde{\mathbf{y}}}(\cdot)$ is a 2-vector and $\Sigma_{\tilde{\mathbf{y}}}(\cdot)$ is a 2×2 matrix:

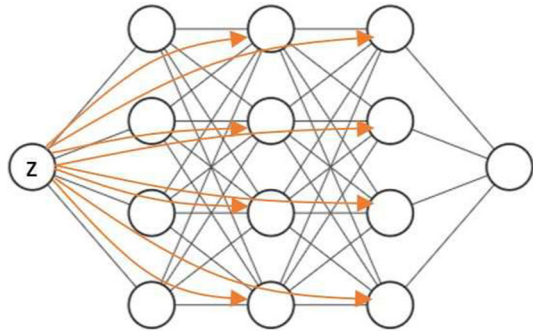
$$\boldsymbol{\mu}_{\tilde{\mathbf{y}}}(t, \tilde{\mathbf{y}}_t) = \begin{pmatrix} r(\tilde{S}_t - a_S) \\ \kappa(b_V\gamma + a_V - \tilde{V}_t) \end{pmatrix}, \quad \Sigma_{\tilde{\mathbf{y}}}(t, \tilde{\mathbf{y}}_t) = \begin{pmatrix} (\tilde{S}_t - a_S)\sqrt{\frac{\tilde{V}_t - a_V}{b_V}}\Sigma_S^\top \\ \sigma\sqrt{b_V(\tilde{V}_t - a_V)}\Sigma_V^\top \end{pmatrix}.$$

The value of the contract is a function of rescaled time and state variables, $L_t = V(\tilde{t}, \tilde{\mathbf{y}}_t | r, \kappa, \gamma, \sigma, \rho, \tilde{T})$. The valuation Eq. (4) is then rewritten in rescaled form as follows:

$$\begin{aligned}0 &= b_h \partial_{\tilde{t}} V - r V + \boldsymbol{\mu}_{\tilde{\mathbf{y}}}(t, \tilde{\mathbf{y}}_t)^\top \nabla_{\tilde{\mathbf{y}}} V \\ &\quad + \frac{1}{2} \text{tr} \left(\Sigma_{\tilde{\mathbf{y}}}(t, \tilde{\mathbf{y}}_t) \Sigma_{\tilde{\mathbf{y}}}(t, \tilde{\mathbf{y}}_t)^\top \mathcal{H}_{\tilde{\mathbf{y}}}(V) \right),\end{aligned}\quad (10)$$

where $\nabla_{\tilde{\mathbf{y}}} V$ and $\mathcal{H}_{\tilde{\mathbf{y}}}(V)$ are respectively the gradient and the Hessian of V with respect to standardized state variables, $\tilde{\mathbf{y}}$. The rescaled version of the terminal and lower

Fig. 1 Feed forward network with skip connections toward intermediate layers



boundary constraints (10) and (6) are:

$$V\left(\tilde{T}, \tilde{\mathbf{y}}_T \mid r, \kappa, \gamma, \sigma, \rho, \tilde{T}\right) = H\left(\frac{\tilde{S}_T - a_S}{b_S}\right), \quad (11)$$

$$V\left(\tilde{t}, \left(a_S, \tilde{V}_T\right)^\top \mid r, \kappa, \gamma, \sigma, \rho, \tilde{T}\right) = e^{-r \frac{(\tilde{T}-\tilde{t})}{b_h}} H(0). \quad (12)$$

The next section explains how to solve Eq. (10) with boundary conditions (11) and (12), using neural networks.

4 Neural networks, European payoffs

We approximate the value function solving Eq. (10) using a particular type of neural network. This network takes as input a vector of dimension 9, denoted by $\mathbf{z}$, containing the scaled time, state variables and parameters:

$$\mathbf{z} := \left(\tilde{t}, \tilde{\mathbf{y}}_t, r, \kappa, \gamma, \sigma, \rho, \tilde{T}\right)^\top. \quad (13)$$

There is no systematic procedure for determining the optimal structure of a neural network. As proven by Hornik (1991), we know that single-layer neural networks can approximate regular functions arbitrarily well, but achieving reasonable accuracy may require a high number of neurons. An alternative approach is provided by feed-forward networks, where information flows forward through several neural layers. However, these networks often struggle to replicate non-linear functions with a limited number of layers. To address this challenge, Sirignano and Spiliopoulos (2018) proposed a variant of LSTM called the Deep Galerkin Network. While this structure replicates prices effectively, its calibration is time-consuming due to the complexity of neural cells.

In our case, we adopt a simpler architecture where intermediate layers receive inputs from the previous layer and the initial input vector. Figure 1 illustrates the structure of such networks, featuring 'skip connections.' This model falls into the category

of residual neural networks, which have been successfully applied in various deep learning applications.

Definition Let $l, n_0, n_1, \dots, n_l \in \mathbb{N}$ be respectively the number of layers and neurons in each layer. n_0 is also the size of input vector. The activation function of layer $k = 1, 2, \dots, l$ is noted $\phi_k(\cdot) : \mathbb{R} \rightarrow \mathbb{R}$. Let $C_1 \in \mathbb{R}^{n_1 \times \mathbb{R}^{n_0}}, \mathbf{c}_1 \in \mathbb{R}^{n_1}, C_k \in \mathbb{R}^{n_k \times \mathbb{R}^{n_0+n_{k-1}}}, \mathbf{c}_k \in \mathbb{R}^{n_k}$ for $k = 2, \dots, l-1, C_l \in \mathbb{R}^{n_l \times \mathbb{R}^{n_{l-1}}}, \mathbf{c}_l \in \mathbb{R}^{n_l}$ be neural weights defining the input, intermediate and output layers. We define the following functions

$$\begin{cases} \psi_k(\mathbf{x}) = \phi_k(C_k \mathbf{x} + \mathbf{c}_k), & k = 1, l \\ \psi_k(\mathbf{x}, \mathbf{z}) = \phi_k\left(C_k \begin{pmatrix} \mathbf{x} \\ \mathbf{z} \end{pmatrix} + \mathbf{c}_k\right), & k = 2, \dots, l-1 \end{cases}$$

where activation functions $\phi_k(\cdot)$ are applied componentwise. The neural network is a function $F : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_l}$ defined by

$$F(\mathbf{z}) = \psi_l \circ \psi_{l-1}(\cdot, \mathbf{z}) \dots \circ \psi_2(\cdot, \mathbf{z}) \circ \psi_1(\mathbf{z}).$$

After having chosen a network architecture, the model is trained by minimizing a loss function that is proportional to errors of approximation. This error is measured by replacing $V(\cdot)$ with $F(\cdot)$ in the scaled FK Eq. (10), at random points in the domain.

At time $t \in [0, T]$, the domain of the state vector, $\mathbf{y}_t = (S_t, V_t)^\top$ is $\mathbb{R}^{+2}$. We approximate this domain by a closed convex subspace:

$$\mathcal{D}^y : [S_l, S_u] \times [V_l, V_u].$$

In order to fit the neural network, we draw a sample of $n_{\mathcal{D}}$ realizations of $\mathbf{y}_t$ in $\mathcal{D}^y$, at random times. We also sample parameters $(r_j, \kappa_j, \gamma_j, \sigma_j, \rho_j, T_j)_{j=1, \dots, n_{\mathcal{D}}}$ under the constraints:

$$\begin{cases} r_j \in [r_l, r_u], \kappa_j \in [\kappa_l, \kappa_u], \\ \gamma_j \in [\gamma_l, \gamma_u], \sigma_j \in [\sigma_l, \sigma_u], \\ \rho_j \in (-1, 1), T_j \in [0, T_{max}], \\ t_j \leq T_j. \end{cases}$$

The set of sampled state variables and parameters is noted $\mathcal{S}_{\mathcal{D}} = (t_j, \mathbf{y}_j, r_j, \kappa_j, \gamma_j, \sigma_j, \rho_j, T_j)_{j=1, \dots, n_{\mathcal{D}}}$. We generate this sample by drawing values between lower and upper bounds, according to a uniform distribution. For the times, we first draw $T_j \in [0, T_{max}]$ and next $t_j \in [0, T_j]$. We next center and scale state variables and times. The mean and standard deviation of sampled state variables are computed with the standard formulas:

$$\bar{\mathbf{y}} = \frac{1}{n_{\mathcal{D}}} \sum_{j=1}^{n_{\mathcal{D}}} \mathbf{y}_j, \quad S_y = \sqrt{\frac{1}{n_{\mathcal{D}} - 1} \sum_{j=1}^{n_{\mathcal{D}}} (\mathbf{y}_j - \bar{\mathbf{y}})^2}.$$

The scaling vectors (7) are defined by $\mathbf{a} = -\frac{\tilde{y}}{\tilde{S}_y}$ and $\mathbf{b} = \frac{1}{\tilde{S}_y}$. For $1, \dots, n_{\mathcal{D}}$, we define $\tilde{\mathbf{y}}_j = \mathbf{a} + \mathbf{b} \odot \mathbf{y}_j$. The times, contract and bond maturities are also rescaled with relations (9) and coefficients

$$a_h = -\frac{1}{2}, \quad b_h = \frac{1}{T_{\max}}.$$

The set of sampled parameters and normalized state variables in the interior domain of the Feynman-Kac equation, is denoted by:

$$\tilde{\mathcal{S}}_{\mathcal{D}} = \left(\tilde{t}_j, \tilde{\mathbf{y}}_j, r_j, \kappa_j, \gamma_j, \sigma_j, \rho_j, \tilde{T}_j \right)_{j=1, \dots, n_{\mathcal{D}}}.$$

During the training phase, the error of approximation is measured for all points of this sample sets with the scaled FK Eq. (11). The error at expiry is measured with another sample set, denoted by $\tilde{\mathcal{S}}_T$, of n_T realizations of state variables and parameters:

$$\tilde{\mathcal{S}}_T = \left(\tilde{\mathbf{y}}_k, r_k, \kappa_k, \gamma_k, \sigma_k, \rho_k, \tilde{T}_k \right)_{k=1, \dots, n_T}.$$

Let us denote by Θ , the vector containing all neural weights $(C_k, c_k)_{k=1, \dots, l}$. At points of $\tilde{\mathcal{S}}_{\mathcal{D}}$, we define for $j = 1, \dots, n_{\mathcal{D}}$, the error in Eq. (10) when V is replaced by the neural network as follows:

$$\begin{aligned} e_j^{\mathcal{D}}(\Theta) &= b_h \partial_{\tilde{t}_j} F - r F + \boldsymbol{\mu}_{\tilde{\mathbf{y}}}(t_j, \tilde{\mathbf{y}}_j)^{\top} \nabla_{\tilde{\mathbf{y}}} F \\ &\quad + \frac{1}{2} \text{tr} \left(\Sigma_{\tilde{\mathbf{y}}}(t_j, \tilde{\mathbf{y}}_j) \Sigma_{\tilde{\mathbf{y}}}(t_j, \tilde{\mathbf{y}}_j)^{\top} \mathcal{H}_{\tilde{\mathbf{y}}}(F) \right). \end{aligned} \quad (14)$$

We refer to $e_j^{\mathcal{D}}$ as the error on the inner domain since it measures the goodness of fit before expiry. The average quadratic loss on $\tilde{\mathcal{S}}_{\mathcal{D}}$ is the first component of the total loss function used to fit the neural network,

$$\mathcal{L}_D(\Theta) = \frac{1}{n_{\mathcal{D}}} \sum_{j=1}^{n_{\mathcal{D}}} e_j^{\mathcal{D}}(\Theta)^2. \quad (15)$$

Since the error $e_j^{\mathcal{D}}(\Theta)$ depends on first and second-order partial derivatives, special attention must be given to the accuracy of their calculation. Computing these derivatives using a standard finite difference method may introduce numerical instabilities. Therefore, we opt for an alternative approach known as automatic or algorithmic differentiation. Automatic differentiation leverages the fact that every computer calculation executes a sequence of elementary arithmetic operations and elementary functions, allowing us to compute partial derivatives with accuracy up to the working precision. Automatic differentiation is implemented in TensorFlow through functions such as GradientTape(.) and tape.gradient(.). For a more in-depth introduction and perspectives, we refer the reader to van Merriënboer et al. (2018).

On the terminal boundary sample set $\tilde{\mathcal{S}}_T$, we define the error $e_k^T(\Theta)$ for $k = 1, \dots, n_T$, as the difference between the output of the neural network and the payoff at expiry:

$$e_k^T(\Theta) = F(\tilde{T}_k, \tilde{y}_k, r_k, \kappa_k, \gamma_k, \sigma_k, \rho_k, \tilde{T}_k) - H\left(\frac{\tilde{S}_k - a_S}{b_S}\right). \quad (16)$$

The average quadratic loss at expiry is the second component of the total loss function.

$$\mathcal{L}_T(\Theta) = \frac{1}{n_T} \sum_{k=1}^{n_T} e_k^T(\Theta)^2. \quad (17)$$

On the lower boundary sample set $\tilde{\mathcal{S}}_{low}$, we define the error $e_l^{low}(\Theta)$ for $l = 1, \dots, n_{low}$, as the difference between the output of the neural network and the discounted payoff at expiry when the stock price reaches its minimum:

$$e_l^{low}(\Theta) = F(\tilde{t}_l, (a_S, \tilde{V}_l)^\top, r_l, \kappa_l, \gamma_l, \sigma_l, \rho_l, \tilde{T}_l) - e^{-r \frac{(\tilde{t}_l - \tilde{t}_l)}{b_h}} H(0).$$

The average quadratic loss on this lower boundary is the third component of the total loss function.

$$\mathcal{L}_{low}(\Theta) = \frac{1}{n_{low}} \sum_{l=1}^{n_{low}} e_l^{low}(\Theta)^2. \quad (18)$$

The optimal network weights are found by minimizing the losses in $\tilde{\mathcal{S}}_D$, $\tilde{\mathcal{S}}_T$ and $\tilde{\mathcal{S}}_{low}$,

$$\Theta_{opt} = \arg \min [\mathcal{L}_D(\Theta) + \mathcal{L}_T(\Theta) + \mathcal{L}_{low}(\Theta)]. \quad (19)$$

In practice, the minimization is performed with a gradient descent algorithm. For an introduction, we refer e.g. to Denuit et al. (2019), Sect. 1.6. Similar to solving partial differential equations using finite differences, there is no universal method for defining the domain of state variables. Their range must be sufficiently broad to confine the propagation of boundary errors within the domain of interest. In the context of market parameters, a narrower range enhances the robustness of the learning process. Expanding the domain of market parameters necessitates a proportional increase in the sizes of the sample sets. $\mathcal{S}_D$, $\mathcal{S}_T$ and $\mathcal{S}_{low}$.

5 Calibration and validation procedures

In the numerical illustration, we will compare put option prices computed by PINN's and Fast Fourier Transform (FFT). There are two alternatives for option pricing by FFT. The first one, proposed by Carr and Madan (2001), inverts numerically the characteristic function of option prices. A single run of this procedure returns option values for a wide range of strike prices. The accuracy of this method nevertheless

depends on a tuning weight (α in the original paper), that ensures the existence of the Fourier transform. The optimal choice of this weight depends on Heston parameters and an inappropriate value generates numerical instabilities. As we need prices for various and randomly generated Heston parameters, tuning the α is problematic. For this reason, we opt for a more robust alternative that approximates the probability density function (pdf) of the stock price by inverting its characteristic function. We briefly recall this method in the next subsection.

5.1 Option pricing by FFT

In the Heston model, the characteristic function of the log-return admits a closed-form expression provided in the next Proposition.

Proposition 1 *The characteristic function of $\ln(S_s/S_0) | \mathcal{F}_t$ under the risk-neutral probability measure $\mathbb{Q}$, for $s \geq t$ with $\omega \in \mathbb{C}$, is given by the following expression*

$$\mathbb{E}^{\mathbb{Q}} \left(e^{\omega \ln(S_s/S_0)} | \mathcal{F}_t \right) = \left(\frac{S_t}{S_0} \right)^{\omega} \exp(A(\omega, t, s) + B(\omega, t, s)V_t). \quad (20)$$

Let us define the following constants:

$$\begin{cases} d = \sqrt{(\rho\sigma\omega - \kappa)^2 + \sigma^2(\omega - \omega^2)}, \\ g = \frac{\kappa - \rho\sigma\omega + d}{\kappa - \rho\sigma\omega - d}. \end{cases}$$

The functions $A(\omega, t, s)$ and $B(\omega, t, s)$ in Eq. (20) are given by

$$A(\omega, t, s) = r\omega(s-t) + \frac{\kappa\gamma}{\sigma^2} \left((\kappa - \rho\sigma\omega + d)(s-t) - 2 \ln \left(\frac{1 - ge^{d(s-t)}}{1 - g} \right) \right), \quad (21)$$

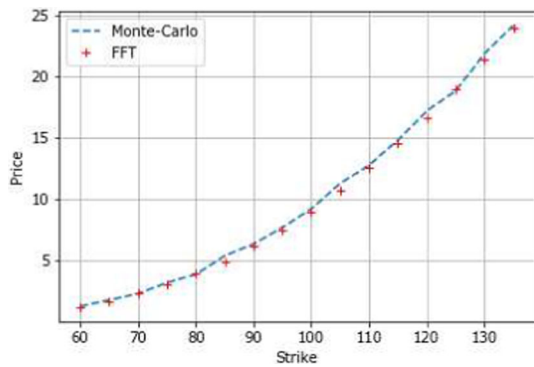
and

$$B(\omega, t, s) = \frac{\kappa - \rho\sigma\omega + d}{\sigma^2} \frac{1 - e^{d(s-t)}}{1 - ge^{d(s-t)}}. \quad (22)$$

For a proof, the reader can refer for instance, to Hainaut (2022), chapter 3, p. 65. European call or put options do not have analytical expressions. In order to evaluate these options, we calculate numerically the probability density function of the log-return by a discrete Fourier transform (DFT). The conditional characteristic function of the log-return given $\mathcal{F}_t$ under $\mathbb{Q}$, $\Upsilon_{t,T}(\omega) = \mathbb{E}^{\mathbb{Q}}(e^{i\omega \ln(S_T/S_0)} | \mathcal{F}_t)$, is also the inverse Fourier transform of its conditional probability density function (pdf):

$$\begin{aligned} f_{t,T}(u) &= \frac{1}{2\pi} \int_{-\infty}^{+\infty} \Upsilon_{t,T}(i\omega) e^{-i\omega u} d\omega \\ &= \frac{1}{\pi} \operatorname{Re} \left(\int_0^{+\infty} \Upsilon_{t,T}(i\omega) e^{-i\omega u} d\omega \right) \end{aligned} \quad (23)$$

Fig. 2 Comparison of FFT and MC prices, 5 years put options



Therefore, we can retrieve the pdf by computing numerically its Fourier transform as stated in the next proposition.

Proposition 2 Let M be the number of steps used in the Discrete Fourier Transform (DFT) and $\Delta_u = \frac{2u_{\max}}{M-1}$ be this step of discretization. Let us denote $\Delta_\omega = \frac{2\pi}{M\Delta_u}$ and

$$\omega_m = (m-1)\Delta_\omega,$$

for $m = 1 \dots M$. Let $\Upsilon_{t,T}(\omega) = \mathbb{E}^{\mathbb{Q}}(e^{\omega \ln(S_T/S_0)} | \mathcal{F}_t)$ be the conditional moment generating function of $\ln(S_T/S_0)$ under $\mathbb{Q}$ given $\mathcal{F}_t$. The values of $f_{i,T}(\cdot)$ the pdf of $\ln(S_T/S_0) | \mathcal{F}_t$ at points $u_k = -\frac{M}{2}\Delta_u + (k-1)\Delta_u$ are approximated by the sum:

$$f(u_k) \approx \frac{2}{M\Delta_u} \operatorname{Re} \left(\sum_{m=1}^M \varrho_m \Upsilon_{t,T}(i\omega_m) (-1)^{m-1} e^{-i\frac{2\pi}{M}(m-1)(k-1)} \right). \quad (24)$$

where $\varrho_m = \frac{1}{2}1_{\{m=1\}} + 1_{\{m \neq 1\}} + \frac{1}{2}1_{\{m=M\}}$.

This result is proven by discretizing the integral (23). The value of a European option of maturity T and payoff $H(S_T)$ is then approximated by the following sum

$$\mathbb{E}^{\mathbb{Q}} \left(e^{-r(T-t)} H(S_T) | \mathcal{F}_t \right) \approx \sum_{k=1}^M f(u_k) H(S_0 e^{u_k}). \quad (25)$$

Prices obtained by this method are compared to PINN prices. This method is sensitive to the choice of $u_{\max}$ and M . By construction, we have the constraint that $\Delta_\omega \Delta_u = \frac{2\pi}{M}$. For a given M , a small discretization step Δ_u for log-returns implies a large discretization step Δ_ω of frequencies and vice-versa. A reasonable accuracy can only be achieved if Δ_ω and Δ_u are small enough. In the numerical illustration, we set $M = 2^8$ and $u_{\max} = 2.2$. To motivate this choice, we compare in Fig. 2 DFT and Monte-Carlo (MC) prices of 5-year put options, for strikes ranging from 60 to 140. Market parameters are reported in Table 3. Notably, the average absolute spread between MC and DFT prices is small, measuring 0.26.

Table 1 Intervals from which parameters and state variables are generated

Range of parameters and state variables	
$S_t \in [20, 180]$	$V_t \in [0.03^2, 0.5^2]$
$r_j \in [0.01, 0.07]$	$\kappa_j \in [0.5, 2]$
$\gamma_j \in [0.06^2, 0.4^2]$	$\sigma_j \in [0.1, 0.9]$
$\rho_j \in [-0.8, 0.8]$	$T_j \in [0, 5]$
$t_j \leq T_j$	

Table 2 Learning rates and number of epochs used in the four training phases

Phase	Learning rate	Epochs
1	0.005	500
2	0.002	1,000
3	0.001	1,000
4	0.0001	1,000

5.2 Learning dataset, calibration procedure and metrics

We estimate several PINN's for pricing European put options of maturities up to 5 years in various market conditions. During the training, the inner error is computed with $n_D = 20,000$ combinations of parameters and state variables. These are randomly drawn from intervals reported in Table 1. The errors on the terminal and lower boundaries are both assessed with sample sets of sizes $n_T = 5000$ and $n_{low} = 5000$. The learning space, $\mathcal{S}^D$, is relatively large and cover various market conditions. Note that we have used the same data sets for all proposed neural architectures for an easy comparison of results.

The option strike is set to $K = 100$ but this is not a limitation. As the payoff is piecewise linear, we use the rule of thumb for valuing options with other strikes. Let us momentarily denote the option price at time t by $V(S_t, K)$, for an asset value S_t and a strike K . The value of an option of strike $K' \neq K$ is then equal to:

$$V(S_t, K') = \frac{K'}{K} V\left(\frac{K}{K'} S_t, K\right),$$

and can therefore be assessed with the network, without retraining. Comparing different networks poses a challenge as their performance is influenced by both network complexity and the number of training epochs. A more complex network typically requires more epochs to attain the same level of accuracy as a simpler one. However, determining the additional training epochs needed for equivalent or superior performance remains elusive. Therefore, we adopt a consistent approach by using the same number of epochs for training different networks. We also take care to apply a calibration protocol that is consistent across all tested configurations. We employ the Adam algorithm to train the networks, a variant of stochastic gradient descent. During the learning phase, the convergence speed is inversely proportional to the learning rate

Table 3 Market parameters for the calculation of the MSE with a single Heston configuration

MSE_{config}			
γ	0.15 ²	r	0.03
κ	0.80	V_0	1.20 γ
σ	0.10	ρ	-0.40

when the network is significantly sub-optimal. As the network approaches optimal values, weight oscillations around these values may occur with larger learning rates. To address this, the training process is divided into four phases, each with a decreasing learning rate and a variable number of iterations. Details regarding the learning rates and number of iterations per phase are provided in Table 2.

Decreasing the learning rate per phase reduces the number of iterations compared to a constant low learning rate. This reduction in the number of iterations shortens the model execution time, enabling relatively rapid calibration. Standardization of the calibration process allows for the comparison of errors at the end of 3,500 iterations. It is noteworthy that for some architectures, a larger number of iterations could potentially result in lower errors but we decided to use the same number of epochs for all networks. The comparison of networks is based on several performance metrics. The first ones are the total loss function and its components $\mathcal{L}_D$, $\mathcal{L}_T$, $\mathcal{L}_{low}$ after training. We also calculate the mean square error, denoted by MSE_{rand} , between PINN and FFT prices. This MSE is computed with a sample set of 5000 random combinations of parameters and state variables. A third metric is the mean square error computed for a fixed set of Heston parameters, reported in Table 3, and random values $S_0 \in [20, 180]$, $T \in [0, 5]$. This error, denoted by MSE_{config} , is valued on a sample set of 5000 pairs (T, S_0) . The last measure is the relative error for in-the-money options (i.e. $S_t < K$). This error is denoted by $Err_{relative}$ and computed on a random set of 5000 combinations of Heston parameters and state variables.

6 Numerical analysis

Tables 4 and 5 respectively report the values of losses and the performance metrics of tested network architectures. We observe that lower losses after 3500 training iterations are obtained with networks having a higher number of layers and neurons. This is consistent with the idea that increasing the number of neural weights allows greater freedom in optimizing the network. For a given number of epochs, the reduction in loss components is therefore greater for complex architectures. Based on the analysis of losses, we would prefer the network with 4 layers and 256 neurons. We also see that the loss on the lower boundary remains small regardless of the architecture because, when S_t is null, the network output is constant and easily replicable with a few neurons, thanks to skip connections. At the beginning of the training, the loss at expiry, $\mathcal{L}_T$, is notably larger than the loss on the inner domain, $\mathcal{L}_D$. During the initial iterations, the primary focus is on tuning neural weights until $\mathcal{L}_T$ and $\mathcal{L}_D$ become comparable.

Table 4 Total loss and its components

# of layers	# of neurons per layer	Total loss	Inner loss $\mathcal{L}_D$	Lower loss $\mathcal{L}_{\text{low}}$	T loss $\mathcal{L}_T$
2	32	0.411	0.234	0.013	0.164
3	32	0.364	0.210	0.011	0.143
4	32	0.289	0.163	0.009	0.117
2	64	0.307	0.183	0.008	0.116
3	64	0.191	0.122	0.004	0.065
4	64	0.139	0.085	0.004	0.051
2	128	0.175	0.103	0.004	0.068
3	128	0.104	0.064	0.002	0.038
4	128	0.066	0.044	0.001	0.021
2	256	0.119	0.067	0.003	0.049
3	256	0.050	0.030	0.001	0.018
4	256	0.049	0.030	0.001	0.017

Table 5 Mean squared errors and relative errors for the tested configurations

# of layers	# of neurons per layer	MSE_{rand}	MSE_{config}	Err_{relative}
2	32	2.611	2.477	0.046
3	32	2.892	2.505	0.043
4	32	2.195	0.924	0.040
2	64	2.877	2.977	0.045
3	64	2.583	1.274	0.045
4	64	2.622	0.907	0.043
2	128	2.583	1.199	0.043
3	128	2.509	1.031	0.044
4	128	2.462	0.700	0.044
2	256	2.206	0.394	0.042
3	256	2.373	0.651	0.044
4	256	2.346	0.689	0.044

This explains why most of networks exhibits similar losses on the inner domain and terminal boundary.

We also see that the 2-layer, 256-neuron network underperforms compared to the 4-layer, 128-neuron counterpart. Two factors contribute to this discrepancy. Firstly, for achieving equivalent accuracy, shallow networks often require a larger number of neurons compared to deep neural architectures. Intuitively, each layer in a deep neural network adds its own level of non-linearity to the preceding layers, whereas in shallow networks, non-linear relations are merely aggregated. Secondly, the 2-layer, 256-neuron and 4-layer, 128-neuron networks contain 139,009 and 72,065 neural

Table 6 Market parameters for the analysis of the 4-256 network

γ	0.20 ²	r	0.04
κ	1.15	ρ	-0.40
σ	0.20		

weights, respectively. Consequently, the stochastic gradient algorithm converges more rapidly to a local minimum when training smaller networks as opposed to larger ones, given the same number of epochs.

Table 5 reveals that the MSE_{rand} 's on a random set, range between 2.195 and 2.892. It seems that complex architectures with a low training loss do not necessarily minimize the MSE on a random sample. For instance, the network with 4 layers and 32 neurons achieves the lowest MSE_{rand} whereas its total calibration error reaches 0.289. We can partly explain this by the tendency of complex networks to overfit data. Ideally, we should increase the size of training sets proportionally to model complexity in order to limit overfitting. However, we haven't adapted the size of the training set to allow for a meaningful comparison. On the other hand, we should not forget that FFT prices are themselves numerical approximations of exact ones. This introduces a non-negligible bias in the calculation of MSE's. Furthermore for some Heston parameters, the FFT method fails to compute a price (due to an overflow encountered in the calculation of the characteristic function). The calculation of prices for these configurations requires a fine tuning of FFT parameters, case by case, which is not possible given the size of the training set. The MSE are computed only with Heston parameters for which the FFT yields a price (i.e. for more than 90% of training data). For the Heston configuration of Table 3, we observe significant variation of MSE_{config} depending on the model complexity. According to this criterion, the best network has 2 layers of 256 neurons. The relative errors for in-the-money options are comparable and close to 4%. To understand from where come errors, we focus on a network with 4 layers of 256 neurons (4-256 network) and study its performance in greater depth.

6.1 Analysis of the 4-256 network, fixed market parameters

In this subsection, we consider a unique set of Heston parameters, presented in Table 6.

The left plot of Fig. 3 compares FFT and PINN prices of 5 years European put when $V_0 = 0.25^2$. We observe that FFT and PINN prices are close regardless of the stock value, S_0 . For high value of S_0 , PINN prices converges to zero at a faster pace than those computed by FFT. This is partly due to the numerical errors of the FFT algorithm.

The right plot of Fig. 3 presents option prices at expiry and reveals the terminal boundary condition is well fulfilled. The left plot of Fig. 4 shows a heatmap of absolute gaps between PINN and FFT option prices for maturities between 0 and 5 years and $S_0 \in [20, 180]$. It confirms that spreads between FFT and PINN prices remain relatively small whatever the maturity and S_0 . For a given maturity, the largest pricing

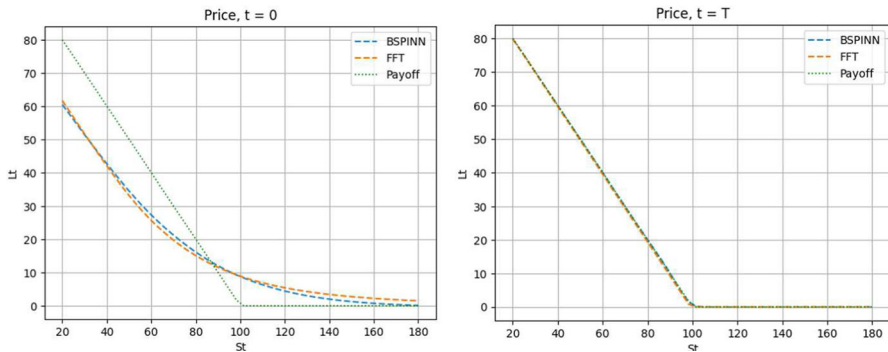


Fig. 3 5 years European put priced with PINN and FFT

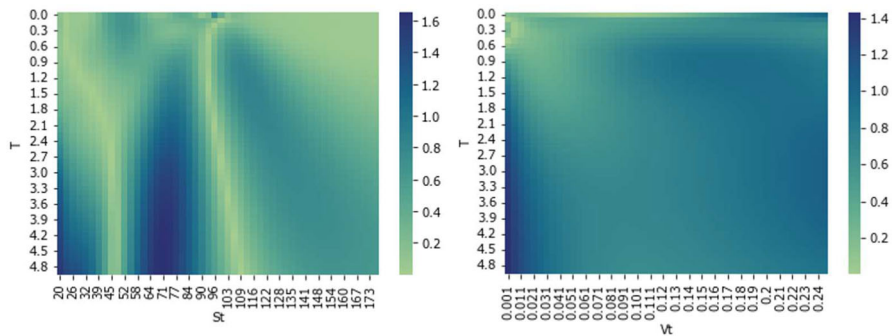


Fig. 4 Heatmaps of absolute differences between PINN and FFT prices

errors are observed in areas where the convexity of the price curve is the highest. The right plot of Fig. 4 presents the heatmap of price spreads for maturities $T \in [0, 5]$ and $V_0 \in [0.001, 0.24]$. The stock value is set to $S_0 = 90$. The spreads are small and nearly constant. We observe a small increase at short term and for higher initial variances.

6.2 Analysis of the 4-256 network, random market parameters

In this subsection, we analyze the spreads between FFT and PINN prices for various Heston parameters. The right plot of Fig. 5 shows the heatmap of absolute differences between FFT and PINN prices by maturities and stock values. To limit computational time, we consider 2500 combinations of Heston parameters and state variables simulated as follows. For each value of S_t ranging from 20 to 180 in steps of 3.2 (50 values), we randomly draw 50 Heston parameters, maturities, and variances. For each combination, we then compute the put price using both FFT and PINN methods. Some of cells of the heatmap are uncolored because the FFT algorithm fails to compute a price for the given set of Heston parameters.

As previously mentioned, the FFT algorithm's performance is affected by the number of discretization steps (M) and the domain of the probability density function (pdf), which is set to $[-u_{max}; u_{max}]$ in this case with $M = 2^8$ and $u_{max} = 2.2$.

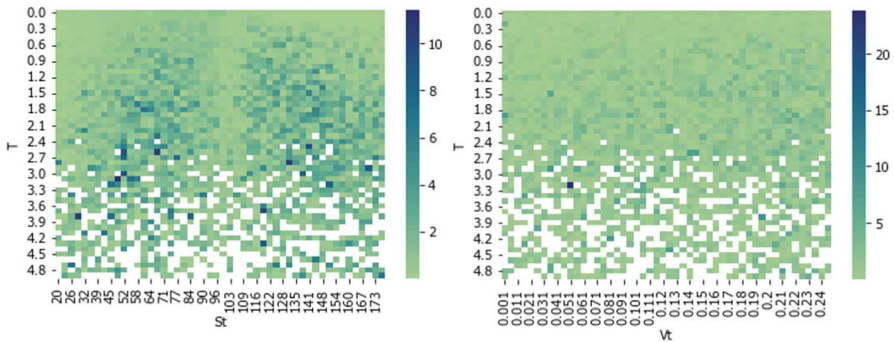


Fig. 5 Heatmap of spreads between PINN and FFT prices on a random sample of parameters. Left plot: T vs S_0 . Right plot: T versus V_0

When the volatility is high, the FFT algorithm fails to compute a price due to an overflow encountered in the calculation of the characteristic function. Obtaining a value requires tuning M and u_{max} on a case-by-case basis without guarantee of accuracy. In comparison the PINN always yields a price regardless of the market parameters and is, from this viewpoint, much more robust than the FFT. On the other hand, we observe that for most standard market configurations, the pricing error is under control. The gap between FFT and PINN price increases with maturity but this increase is partly attributed to the numerical instability of the FFT method. The left plot of Fig. 5 shows the same heatmap with respect to maturities and variances (2500 points). This confirms that the PINN pricing error is reasonable across most of the configurations.

6.3 Sensitivity of the 4-256 network to training parameters

In this section, we analyze the sensitivity of performance metrics with respect to parameters of the training procedure. We focus on the network with 4 layers and 256 neurons. We perform three tests:

- **Test A:** Increase of the sample size $n_{\mathcal{D}}$ from 20,000 to 100,000 items and n_T, n_{low} from 5000 to 25,000.
- **Test B:** Increase of the number of epochs from 3500 to 6500 and a smaller learning rate of 1/10,000 during the last 3000 iterations.
- **Test C:** Both increases of the sample size and number of epochs (combination of tests A and B).

The performance metrics after training are reported in Table 7. Several conclusions can be drawn from these tests. Firstly, increasing the size of the training sample (test A: 150,000 in total vs. 30,000 considering inner and boundary data sets) does not result in a significant reduction in the loss function. Furthermore, there is no significant reduction in MSE. Secondly, increasing the number of epochs and reducing the learning rate (test B) allows for a slight reduction in the loss function, but there is no significant reduction in MSE. Thirdly, increasing both the sample size and the number of epochs (test C) slightly reduces the total loss, but MSE does not show improvement. Finally,

Table 7 Performance metrics for various training parameters

	Total loss	Inner loss $\mathcal{L}_D$	Lower loss $\mathcal{L}_{\text{low}}$	T loss $\mathcal{L}_T$	MSE_{rand}	MSE_{config}	Err_{relative}
Initial	0.049	0.030	0.001	0.017	2.346	0.696	0.044
Test A	0.058	0.034	0.001	0.023	2.418	0.667	0.042
Test B	0.037	0.023	0.001	0.014	2.381	0.702	0.043
Test C	0.040	0.024	0.0004	0.016	2.460	0.697	0.044

for the 4-256 network, there is no need to increase the size of the data sample or the number of epochs. With 3500 iterations and 30,000 data items, we achieve optimal accuracy for this network.

6.4 Release of the lower boundary condition

The analysis of inner losses in Table 4 reveals that errors on the lower boundary (when $S_t = 0$) are negligible (< 0.001). Put option prices are constrained to discounted strike prices on this boundary, and this constraint is well integrated by nearly all models. This raises the question of whether we can relax this boundary condition. Our objective is to investigate if ignoring this condition and indirectly assigning it more importance during training could improve the inner and terminal losses. We test this hypothesis for the network with 4 layers and 256 neurons, and the performance metrics are reported in Table 8. The results confirm our intuition: excluding the loss on the lower boundary significantly reduces the inner and terminal losses by -62% . We attribute this to the gradient descent algorithm having more degrees of freedom to minimize $\mathcal{L}_D$ and $\mathcal{L}_T$. The trade-off for this improvement, is the local increase in errors for options valued near the lower boundary. This decrease of accuracy is mirrored by the MSE's, which are not reduced compared to those obtained when including the loss on the lower boundary.

7 Conclusions

This article demonstrates that a Physics-Inspired Neural Network (PINN) is an efficient alternative to standard methods for pricing options. In the financial industry, rapid pricing is of paramount importance for the continuous recalibration of models to market quotes. Furthermore, the constant fluctuations in market parameters call for a robust tool. PINNs provide a solution to these challenges. After training, valuing an option with a PINN is indeed quasi-instantaneous and by construction, the PINN is not subject to numerical instabilities.

We focus on the Heston model in this work, a standard framework for option pricing. Utilizing Itô calculus and non-arbitrage arguments, we derive the Feynman-Kac equation governing the prices of financial contingent claims. We propose a scaled and centered version of this partial differential equation (PDE), whose solution is

Table 8 Performance metrics for various training parameters

Lower boundary	Total loss	Inner loss $\mathcal{L}_D$	Lower loss $\mathcal{L}_{\text{low}}$	T loss $\mathcal{L}_T$	MSE_{rand}	MSE_{config}	Err_{relative}
With	0.049	0.030	0.001	0.017	2.346	0.696	0.044
Without	0.019	0.011	n.a.	0.008	2.481	0.699	0.045

approximated by a neural network. We consider a specific type of feedforward neural network with skip connections to intermediate layers to capture the non-linearity of prices.

The calibration of this network does not require exact option values; instead, we minimize, via stochastic gradient descent, the errors when replacing the option value with the Physics-Inspired Neural Network (PINN) in the Feynman-Kac PDE. In comparison to existing literature, the novelty of our approach lies in the network's ability to evaluate options for a wide range of Heston parameters and maturities. Conversely, the Fast Fourier Transform (FFT) method needs to be rerun after every modification of model parameters or option maturity, making PINN a significantly faster pricing process.

As seen in the numerical illustrations, the FFT method is not always numerically stable and requires fine-tuning. The PINN does not suffer from this drawback. However, our approach has practical limitations, especially regarding the structure of the neural network architecture. To the best of our knowledge, no theory on the architecture of the network has yet been formulated for solving stochastic differential equations. We have thus opted for a layer-constant network structure, although other architectural methods (top-down, bottom-up architecture, etc.) could be considered.

On the other hand, the calibration procedure (number of iterations, learning rate, choice of optimizer) relies more on empirical practices than on theoretical results. Empirical use is crucial for choosing the right calibration, and a trial-and-error phase is necessary in practice. While the performance of neural networks is usually significant, the explicability of models is low. The black-box aspect of this approach limits model comprehension and explicability. A market parameter sensitivity approach can be used to understand the main principles of the market.

The consistency and accuracy demonstrated by Physics-Inspired Neural Networks (PINNs) in valuing European options within the Heston model suggest their potential relevance for pricing more complex exotic derivatives. An extension of the proposed framework to value Bermudian options, for instance, could be explored. Handling multiple exercise dates in this context would require incorporating into the loss function intermediate errors related to early exercises. Further enhancements to the accuracy of PINNs could be achieved through various avenues, such as incorporating long-short term memory cells or employing boosting techniques. Finally, we could imagine to extend the proposed framework to the valuation of American-style options. In this case, we would need a second network for modeling the free-boundary triggering the early exercise of the option.

Acknowledgements The first author thanks the FNRS (Fonds de la recherche scientifique) for the financial support through the EOS project Asterisk (research grant 40007517).

References

- Al-Aradi, A., Correia, A., Jardim, G., de Freitas Naiff, D., Saporito, Y.: Extensions of the deep Galerkin method. *Appl. Math. Comput.* **430**, 127287 (2022)
- Barigou, K., Delong, L.: Pricing equity-linked life insurance contracts with multiple risk factors by neural networks. *J. Comput. Appl. Math.* **404**, 113922 (2022)

- Beck, C., Weinan, E., Jentzen, A.: Machine learning approximation algorithms for high dimensional fully nonlinear partial differential equations and second-order backward stochastic differential equations. *J. Nonlinear Sci.* **29**, 1563–1619 (2019)
- Biagini, F., Gonon, L., Reitsam, T.: Neural network approximation for superhedging prices. *Math. Finance* **33**(1), 146–184 (2023)
- Buehler, H., Gonon, L., Teichmann, J., Wood, B.: Deep hedging. *Quant. Finance* **19**(8), 1–21 (2019)
- Cai, Z.: Approximating quantum many-body wave-functions using artificial neural networks. *Phys. Rev. B* **97**, 035116 (2018)
- Cai, S., Wang, Z., Wang, S., Perdikaris, P., Em, Karniadakis G.: Physics-informed neural networks for heat transfer problems. *J. Heat Mass Trans.* **143**(6), 060801 (2021)
- Carr, P., Madan, D.: Option valuation using the fast Fourier transform. *J. Comput. Finance* **2**(4), 61–73 (2001)
- Cuomo, S., Schiano Di Cola, V., Giampaolo, F., Rozza, G., Raissi, M., Piccialli, F.: Scientific machine learning through physics-informed neural networks: where we are and what's next. *J. Sci. Comput.* **92**, 88 (2022)
- Carleo, G., Troyer, M.: Solving the quantum many-body problem with artificial neural networks. *Science* **355**(6325), 602–606 (2017)
- Delong, L., Dhaene, J., Barigou, K.: Fair valuation of insurance liability cash-flow streams in continuous time: theory. *Insur. Math. Econ.* **88**, 196–208 (2019)
- Denuit, M., Trufin, J., Hainaut, D.: Effective statistical learning for actuaries III. Springer Nature, NY (2019)
- Doyle, D., Groendyke, C.: Using neural networks to price and hedge variable annuity guarantees. *Risks* **7**(1), 1 (2019)
- Friedman, A.: Partial differential equations of parabolic type. Prentice-Hall, Englewood Cliffs, N.J. (1964)
- Friedman, A.: Stochastic differential equations and applications. Academic Press, New York (1975)
- Gatta, F., Di Cola, V.S., Giampaolo, F., Piccialli, F., Cuomo, S.: Meshless methods for American option pricing through physics-informed neural networks. *Eng. Anal. Bound. Elem.* **151**, 68–82 (2023)
- Germain, M., Pham, H., Warin, X.: Neural networks-based algorithms for stochastic control and PDE's in finance. (2021) [arXiv:2101.08068](https://arxiv.org/abs/2101.08068)
- Glaw, K., Wunderlich, L.: The deep parametric PDE method and applications to option pricing. *Appl. Math. Comput.* **432**, 127355 (2022)
- Hainaut, D.: Valuation of guaranteed minimum accumulation benefits (GMABs) with physics-inspired neural networks. *Annals Actuar. Sci.* (2024). <https://doi.org/10.1017/S1748499524000095>
- Hainaut, D.: Continuous time processes for finance. Switching, Self-exciting, fractional and other recent dynamics. Springer & Bocconi Series in mathematics, statistics, finance and Economics (2022)
- Hejazi, S.A., Jackson, K.R.: A neural network approach to efficient valuation of large portfolios of variable annuities. *Insur. Math. Econ.* **70**, 169–181 (2016)
- Hornik, K.: Approximation capabilities of multilayer feedforward networks. *Neural Netw.* **4**(2), 251–257 (1991)
- Horvath, B., Teichmann, J., Zuric, Z.: Deep hedging under rough volatility. *Risk* **9**(7), 138 (2021)
- Jiang, Q., Sirignano, J., Cohen, S.: Global convergence of Deep Galerkin and PINNs method for solving PDE. (2023)
- Jin, X., Cai, S., Li, H., et al.: NSFnets (Navier-Stokes flow nets): physics-informed neural networks for the incompressible Navier-Stokes equations. *J. Comput. Phys.* **426**(109), 951 (2021)
- Karatzas, I., Schreve, S.E.: Brownian motion and stochastic calculus, 2nd edn. Springer Science & Business Media, New York (1998)
- Lee, H., Kang, I.S.: Neural algorithm for solving differential equations. *J. Comput. Phys.* **91**(1), 110–131 (1990)
- Mao, Z., Jagtap, A.D., Karniadakis, G.E.: Physics-informed neural networks for high-speed flows. *Comput. Methods Appl. Mech. Engrg.* **360**(112), 789 (2020)
- Raissi, M., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **378**, 686–707 (2019)
- Sirignano, J., Spiliopoulos, K.: DGM: a deep learning algorithm for solving partial differential equations. *J. Comput. Phys.* **375**, 1339–1364 (2018)
- van Merriënboer, B., Breuleux, O., Bergeron, A.: Automatic differentiation in ML: where we are and where we should be going. *Adv. Neural Info. Process. Syst.* **31**, 1–11 (2018)

Weinan, E., Han, J., Jentzen, A.: Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations. *Commun. Math. Stat.* **5**(4), 349–380 (2017)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.